

Examen final de laboratorio

Crea un fichero que se llame "respuestas.txt" donde escribirás las respuestas para los apartados de los ejercicios del control. Indica para cada respuesta, el número de ejercicio y el número de apartado (por ejemplo, 1.a). **Justifica todas tus respuestas. Una respuesta sin justificación se considerará errónea.**

Importante: para cada ejercicio tienes que partir de la versión de Zeos original que te hemos suministrado.

1. (5 puntos) Mailbox

Queremos implementar un sistema para enviar y recibir información entre procesos de usuario. El sistema tiene que ofrecer un buzón (*mailbox*) que permita a los procesos guardar y recuperar mensajes de tamaño variable. Un buzón es una zona de memoria dentro del sistema. Para simplificar puedes suponer que el buzón tiene una capacidad máxima de 4096 bytes.

Para acceder a este buzón hay que implementar las siguientes llamadas a sistema:

```
int sendtombx(void *addr, unsigned int size)
```

Esta función permite guardar un nuevo mensaje en el buzón. El mensaje empieza en la dirección lógica *addr* del proceso y tiene un tamaño de *size* bytes. El tamaño del mensaje debe ser inferior a la capacidad total del buzón. Si el mensaje no cabe en el buzón, el proceso tiene que bloquearse hasta que haya suficiente espacio. Si el buzón tiene otros procesos pendientes de enviar mensajes, también debe bloquear al proceso actual. Esta función devuelve -1 en caso de error o 0 si se ha guardado el mensaje en el buzón.

```
int recvfrommbx(void* addr, unsigned int *size)
```

Esta función permite recuperar el primer mensaje del buzón. El mensaje se guarda en la dirección de usuario *addr*. El parámetro *size* es una variable de entrada/salida que inicialmente contiene el tamaño que el usuario ha reservado para guardar el mensaje; y una vez el mensaje se ha leído, el parámetro se actualizará con el tamaño real del mensaje. Una vez leído, el mensaje entero se borra del buzón. Si el usuario ha reservado menos espacio que el tamaño del mensaje, el mensaje recibido se truncará a ese espacio reservado. Después de leer un mensaje, si el buzón tiene procesos pendientes de enviar mensajes, esta operación debe desbloquear al primer proceso bloqueado. Si no hay ningún mensaje en el buzón cuando se ejecuta esta función, el proceso debe bloquearse hasta que llegue un nuevo mensaje. El orden de las peticiones debe seguir una política FIFO. Esta rutina devuelve -1 en caso de error o el identificador del proceso que ha enviado el mensaje en caso de éxito.

Estas llamadas a sistema **deben ser solamente accesibles** a través de la interrupción 0x90 e identificarse con los números 1 y 2 respectivamente.

- a) Indica para cada una de las llamadas a sistema qué errores se pueden producir. A partir de ahora (y en el código a resolver) puedes suponer que no habrá errores.
- b) Escribe el código del wrapper del *sendtombx*
- c) Escribe el código del handler de la rutina *sendtombx*.
- d) Indica los cambios necesarios en las estructuras del sistema para añadir estas dos nuevas llamadas.
- e) ¿Qué información debes guardar para cada mensaje en el buzón?
- f) ¿Qué campos necesita tener la estructura buzón? Justifica cada uno de ellos.
- g) Implementa la función *bLockThis(&queue, at_end)* que bloquea el proceso actual al final de la cola *queue* si *at_end* vale 1 o al principio en caso contrario.
- h) Asumiendo que tienes una función *copy2mbx(addr, size)* que mágicamente copia el buffer de usuario en el buzón, implementa la función *sys_sendtombx*.
- i) Asumiendo que tienes una función *copymbxTo(addr, size)* que mágicamente copia el primer mensaje del buzón en el buffer de usuario (truncando si es necesario) y devuelve el identificador del proceso que ha enviado el mensaje, implementa la función *sys_recvfrommbx*.
- j) Con todas las restricciones del enunciado y dados 3 procesos: 1 que escribe dos mensajes en el buzón y los otros 2 procesos que intentan leer de este buzón continuamente ¿se puede garantizar el orden en que los procesos leerán los mensajes?

2. (2 punto) Memoria dinámica

En el código del examen queremos añadir una nueva funcionalidad para crear nuevas regiones de memoria en el espacio de usuario. En concreto esta nueva llamada a sistema:

```
void * new_zone(void)
```

Esta funcionalidad crea una nueva región de memoria de 4096 bytes en el espacio de direcciones del proceso actual y devuelve la dirección inicial de esta región. Si no hay más memoria disponible debe devolver una dirección 0.

Estas regiones deben ser heredadas al crear un nuevo proceso.

Para eliminar una de estas regiones usaremos la nueva llamada a sistema:

```
int free_zone(void* addr)
```

Que libera la región que comienza en la dirección *addr* (previamente asignada con la llamada a sistema anterior).

Estas llamadas a sistema **deben ser solamente accesibles** a través de la interrupción 0x90 e identificarse con los números 3 y 4 respectivamente.

Se valorará la simplicidad de la solución.

- a) ¿Qué nuevas estructuras de datos tienes que añadir?
- b) Implementa la función *findFreeLogicalZone* que busca una región disponible en el espacio lógico del proceso.
- c) Implementa *sys_new_zone*.
- d) Implementa *sys_free_zone*.
- e) Indica que cambios habría que hacer en otras partes del sistema.

3. (1 punto) *copyFrom*

Implementa la función *copyFrom(PCB, addr, size)* que copia el rango de direcciones *[addr, addr+size)* del proceso *pid* en el mismo rango del proceso actual. La dirección *addr* debe estar alineada a página. Para realizar la copia puedes suponer que el rango de direcciones *[0x3FF000, 0x400000)* está libre.

4. (2 punto) *Context switch*

El reputado investigador BakaBaka propone el siguiente código en ensamblador para hacer el cambio de contexto y ahorrarnos el wrapper visto en teoría:

```
1:      ; void task_switch(struct task_struct *new)
2:      task_switch:
3:          pushl %ebp
4:          movl %esp, %ebp
5:          pushl %edi
6:          pushl %esi
7:          pushl %ebx
8:          pushl XXX ( %esp ) ; Apilar 'new'
9:          call save_EBP ; Función que guarda el contenido del registro EBP
10:         ; en el campo KERNEL_ESP del PCB del proceso actual
11:          call change_next_stack ; Función que actualiza la pila de sistema a
12:         ; usar la próxima vez que se entre a sistema
13:          call change_memory; Función que actualiza el espacio @para que use
14:         ; el del proceso pasado como parámetro
15:          call get_next_EBP ; Función que devuelve el valor guardado en el
16:         ; campo KERNEL_ESP del PCB pasado como parámetro
17:          movl %eax, %esp
18:          popl %ebx
19:          popl %esi
20:          popl %edi
21:          popl %ebp
22:          ret
```

Responde justificadamente a estas preguntas:

- a) Muestra el contenido de la pila de sistema justo al empezar la rutina *task_switch* implementada originalmente, junto con los comandos que has usado para obtenerlo.
- b) Modifica la compilación para que el fichero con la rutina *inner_task_switch* se compile sin optimización.

SOA2 (15/01/2026)

- c) ¿Qué valor hay que colocar en XXX para apilar el parámetro de la función *task_switch*?
- d) ¿Qué detalle hay que cambiar para que funcione este código? Indica el código necesario para solventarlo.
- e) Indica el código necesario para implementar la función *save_EBP*.
- f) Indica como debe cambiar el contexto del proceso idle en su inicialización.

Entrega

Sube al Racó los ficheros “respuestas.txt” y el código que hayas creado.

Para entregar el código, si lo has hecho, utiliza:

➤ `tar zcvf final.tar.gz directorio_zeos`

SOLUCION

EXERCICI 1

- a) Indica para cada una de las llamadas a sistema qué errores se pueden producir.

- parametros incorrectos: region de usuario (addr, addr+size) fuera del espacio de direcciones
- size > 4096

- b) Escribe el código del wrapper del *sendtombx*

```
eax = 1
ebx = addr
ecx = size
int 0x90
if (eax < 0) { errno = eax, eax = -1 }
return eax
```

- c) Escribe el código del handler de la rutina *sendtombx*.

```
myhandler:
saveall,
check eax,
call NEWSyscalltable[ebx]
mov eax, CTXT_EAX
restoreall
iret
```

- d) Indica los cambios necesarios en las estructuras del sistema para añadir estas dos nuevas llamadas.

```
NEWSyscall:
.long sys_nisyscall
.long sys_sendtombx
.long sys_recvfrommbx

setTrapHandler(0x90, myhandler, 3)

MBX mbx; //The mailbox structure. Requires to initialize its fields.
```

- e) ¿Qué información debes guardar para cada mensaje en el buzón?

```
struct sms{
pid : identificador del proceso que ha enviado
size : tamaño del mensaje
data: contenido del mensaje}
```

- f) ¿Qué campos necesita tener la estructura buzón? Justifica cada uno de ellos.

```
MBX{
    data: char[4096+2*sizeof(int)]: Bytes del mensaje (+ tamaños) Buffer circular
    capacity: int    Capacidad libre del mbx
    READ: int        1ª posicion a leer
    WRITE: int       1ª posición a escribir
    writers: struct list head Lista de procesos bloqueados para enviar
    readers: struct list head Lista de procesos bloqueados para recibir
}
```

- g) Implementa la función *blockThis(&queue, at_end)* que bloquea el proceso actual al final de la cola *queue* si *at_end* vale 1 o al principio en caso contrario.

```
Quitandolo de la cola de run, en este caso colocandolo en la cola de procesos bloqueados en el buzón

blockThis(struct list_head* l, int at_end) {
    if (at_end)
        list_add_tail(current()->list, l);
    else
        list_add(current()->list, l);
    sched_next(); }
```

- h) Asumiendo que tienes una función *copy2mbx(addr, size)* que mágicamente copia el buffer de usuario en el buzón, implementa la función *sys_sendtombx*.

```
if (!list_empty(&mbx->writers)) //Block if writers
    blockThis(&mbx->writers, 1);
while(mbx->capacity < (size + sizeof(int)*2)){    //wait for enough capacity with FIFO
    blockThis(&mbx->writers, 0);
}
copy2mbx(addr,size)
if (!list_empty(&mbx->readers)) // unblock first reader
    update_process_state( list_head_to_task_struct(list_first(&mbx->readers)),
    &readyqueue)
```

- i) Asumiendo que tienes una función *copymbxTo(addr, size)* que mágicamente copia el primer mensaje del buzón en el buffer de usuario (truncando si es necesario), implementa la función *sys_recvfrommbx*

```
while(mbx->capacity == sizeof(mbx->data)){    //wait while mbx is empty
    blockThis(&mbx->readers, 1);
}
PID = copymbxTo(addr,size)
if (!list_empty(&mbx->writers)) // unblock first writer
    update_process_state( list_head_to_task_struct(list_first(&mbx->writers)),
    &readyqueue)
return PID
```

- j) Con todas las restricciones del enunciado y dados 1 proceso que escribe dos mensajes en el buzón y 2 procesos que intentan leer de este buzón continuamente ¿se puede garantizar el orden en que los procesos leerán los mensajes?

No, ya que no hay nada que distinga el destinatario del mensaje, el primero que lea leerá el mensaje, de hecho puede darse el caso que un mismo proceso lea los dos mensajes

EXERCICI3

3A

ninguna, ya que se puede utilizar la propia tabla de paginas para guardar las paginas mapeadas

3B

```
findFreeLogicalZone(){
    pt = getPT(current());
    for(i=0x3FF; i<0x11B; i--) {
        if (pt[i].present == 0) return i;
    }
    return -1; //not found
}
```

3C

```
sys_new_zone() {
    int p = findFreeLogicalZone();
    if (p<0) return (void*) 0; //error
    int n = alloc_frame();
    if (n==0) return (void*) 0; //error
    set_ss_page(getPT(current()), p, n)
    return p<<12;
}
```

3D

```
sys_free_zone(addr) {
    if (addr>>12 < 0x11B) return -1;
    if (addr%4096 >0) return -1;
```

```
PT = getPT(current());
i = addr>>12;
if (!PT[i].present) return -1;
free_frame(get_frame(PT, i));
del_ss_pag(PT, i);
set_cr3(getDIR(current()));
return 0;
}
```

3E

Cambios:

- exit. Liberar todas las zonas reservadas
- fork. Recorrer TP copiando las nuevas zonas al nuevo proceso

EXERCICI2

```
copyFrom(proc, addr, size) {
    numpages = size/4096;
    remain = size%4096;
    numpages++;
    for (int i=0< i<numpages; i++) {
        set_ss_pag(getPT(current()), 0x3FF, get_frame(getPT(proc), page+i);
        if (i < (numpages-1)) //copy just remaining bytes in last iteration
            copy_data(0x3FF<<12, (page+i)<<12, 4096);
        else
            copy_data(0x3FF<<12, (page+i)<<12, remain);
        del_ss_pag(getPT(current()), 0x3FF);
        set_cr3(getDIR(current()));
    }
}
```

EXERCICI 4

- 4.a GDB\$ b task_switch; c; pila
- 4.b sacar el -02 del sched.o
- 4.c XXX = 20 (ebp + ebx + esi + edi + @ret)
- 4.d EBP està mal seteado, mover la linea 4 entre la 7 y la 8.
- 4.e

SOA2 (15/01/2026)

```
// En C
    save_ebp(void) {
        int ebp = getEBP();
        current()->kernel_esp = ebp;
    }

    getEBP:
        mov %EBP, %eax
        ret

// En ASM (calculando offset del campo kernel_esp = 16)
    save_ebp:
        mov %ebp, %ecx
        call current
        mov %ecx, 16(%eax)
        ret
4.e La inicialización del proceso idle debe quedar:
    dummy : dummy: dummy : dummy: @cpu_idle
```